

RAGAS Multi-Tenant Console

Architecture & rollout plan – extending the existing evaluator into a company-wide platform

PREPARED FOR	DATE	REV	SCOPE
Client walkthrough	05 Aug 2026	B — for review	Planning only

A-00 · Overview A-01 · Roles A-02 · Provisioning A-03 · Evaluate & report A-04 · Archite

A-00

What we're building

OVERVIEW

Today RAGAS runs as a single-user Streamlit app: one person points it at a RAG endpoint and a judge LLM, runs an evaluation, and reads the Allure report.

This plan turns that into a console your team can hand to clients. A **Main Admin** at your company onboards each client as a **Project**. That project's owner — the **Project Admin** — creates logins for their own team, the **Project Users**. Every signed-in project member — admin or user — saves their own RAG + judge-LLM credentials and runs evaluations themselves; the report they produce is shared with the whole project, not kept to themselves.

No code changes yet. This document lays out the roles, the data model, the screens, and the Azure/Postgres architecture, so it can be reviewed — internally and with the client — before anything is built. Two tags appear throughout:

RECOMMENDED a default we suggest, and **OPEN DECISION** a place we made an assumption and want your nod (collected in full at A-09).

What's new in Rev B — based on the follow-up that this isn't just a report viewer:

- Any Project User or Project Admin can now save their own RAG + judge-LLM API key and *run* evaluations, not just view a report someone else ran (A-03).
- Every run *appends* to the project's shared report, tagged with who ran it — nothing is silently replaced (A-03, A-06).

- Reports stay strictly per-project as they grow; "Clear report" is now an explicit, separate Project Admin action (A-07).
- Two new tables — `user_configs` and `evaluation_runs` — carry the per-user credentials and the run history (A-05).

3

ROLES

Main Admin, Project Admin, Project User — one login table, one permission model.

1

SHARED ENGINE

The existing evaluation pipeline and Allure reporting are reused per run, not rebuilt.



CUMULATIVE

Every run appends to the project's report, tagged with who ran it.

Az

POSTGRES + AZURE

Your stated stack: Azure Database for PostgreSQL, Azure App Service, Azure DevOps.

A-01

Roles & permissions

ACCESS MODEL

ROLE	CAN DO	CANNOT DO
Main Admin	Add a Project (name, owner, owner email); see every project and the total project count on the Dashboard; open any project to view its user list.	Cannot create a project's users directly, and doesn't run evaluations — this role is provisioning only.
Project Admin	Create/manage usernames & passwords for their project; save their own RAG + judge-LLM config and run evaluations; view the project's full, cumulative report; clear it.	Cannot see or manage other projects; cannot create new projects; cannot read another user's saved API key.
Project User	Save their own RAG + judge-LLM config and run evaluations; view the project's full, cumulative report — every run, by every member, not just their own.	Cannot create users; cannot clear the report.

By design: only the Main Admin creates Projects, and only a Project Admin creates that project's Users. Every password is set directly by whoever creates the account — no self-service sign-up, no "forgot password" yet (future scope, see A-09). Running an evaluation and viewing the report are shared abilities — Admin and User differ only in user-management and the clear action.

A-02

Provisioning flow

ONBOARDING

How a client goes from a name on a call to a logged-in user — three lanes, one handoff at a time. What each person does *after* logging in is covered separately in A-03.

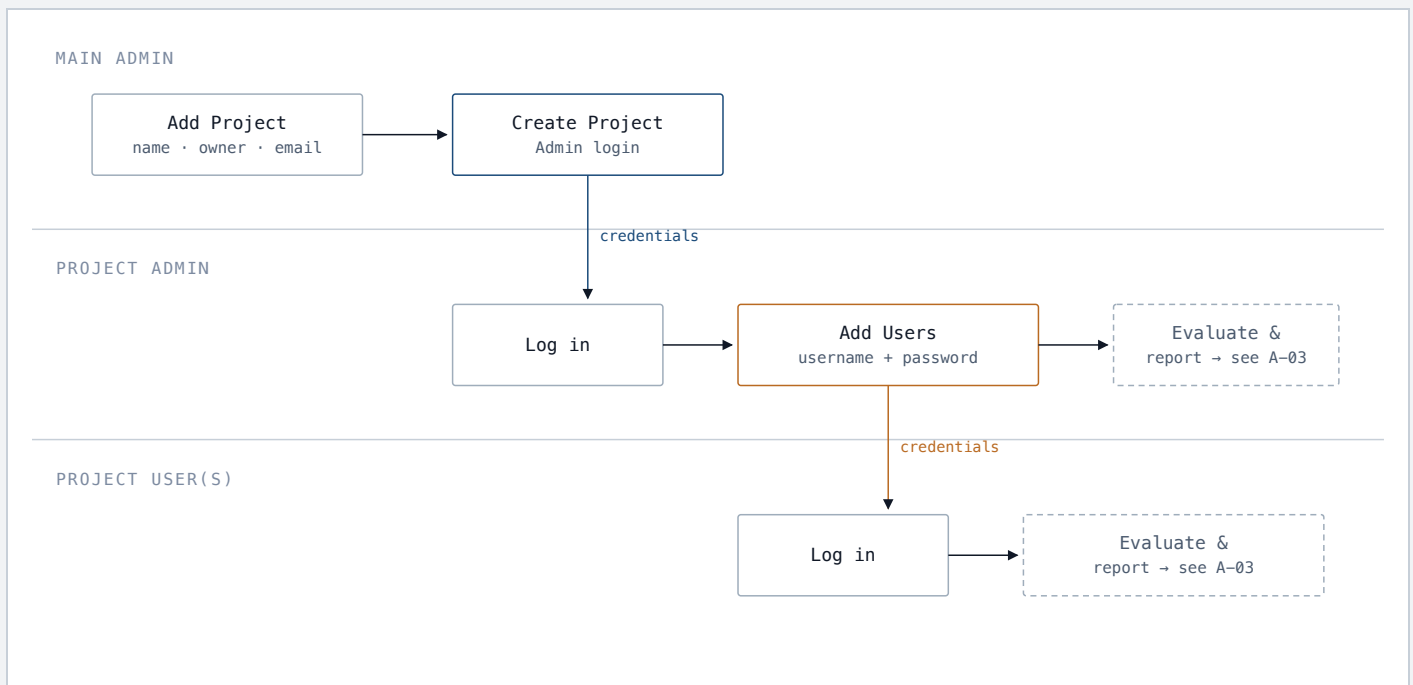


Fig. A-02 — Provisioning is a single chain of custody: Main Admin → Project Admin → Project Users, with credentials handed down one level at a time. Dashed boxes hand off to A-03.

OPEN DECISION We assumed the Main Admin sets the *first* Project Admin login at the moment the project is created, bootstrapped from the owner name + email captured on "Add Project." Confirm this matches intent — full list at A-09.

A-03

Evaluate & report flow

NEW IN REV B

This is the part that changed: it's not one shared config and a report you only watch. Every project member — Admin or User — brings their *own* RAG + judge-LLM credentials, runs the evaluation themselves, and their results land in one growing, shared report for the project.

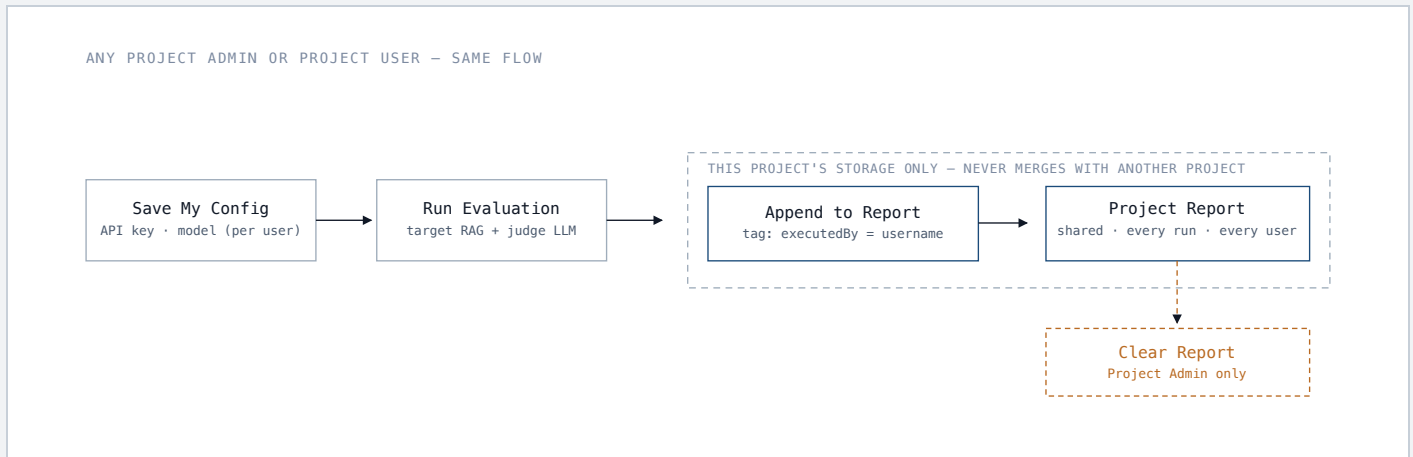


Fig. A-03 — The dashed outer box is the isolation guarantee made visible: append and report both resolve to `{project_id}/allure-results/` from the caller's own session, so a run can never land in — or be read from — a project the caller doesn't belong to. Clearing (bottom right, brass) is the one destructive, admin-only branch off this path.



VISIBLE TO ALL

The report shows every run, run by anyone on the project — not just the report's own author.



APPENDED, NOT REPLACED

New runs add to the history; older runs stay until a Project Admin explicitly clears it.



CONFIG STAYS PERSONAL

Each member's own API key is never shown to teammates, including their Project Admin.

A-04

System architecture

AZURE + POSTGRES

RECOMMENDED

Extend the existing Streamlit app rather than rebuild it. Its evaluation engine, Allure integration, and config loader already work — a new auth/RBAC

layer, per-user config storage, and the append logic sit alongside them, not underneath a rewrite.

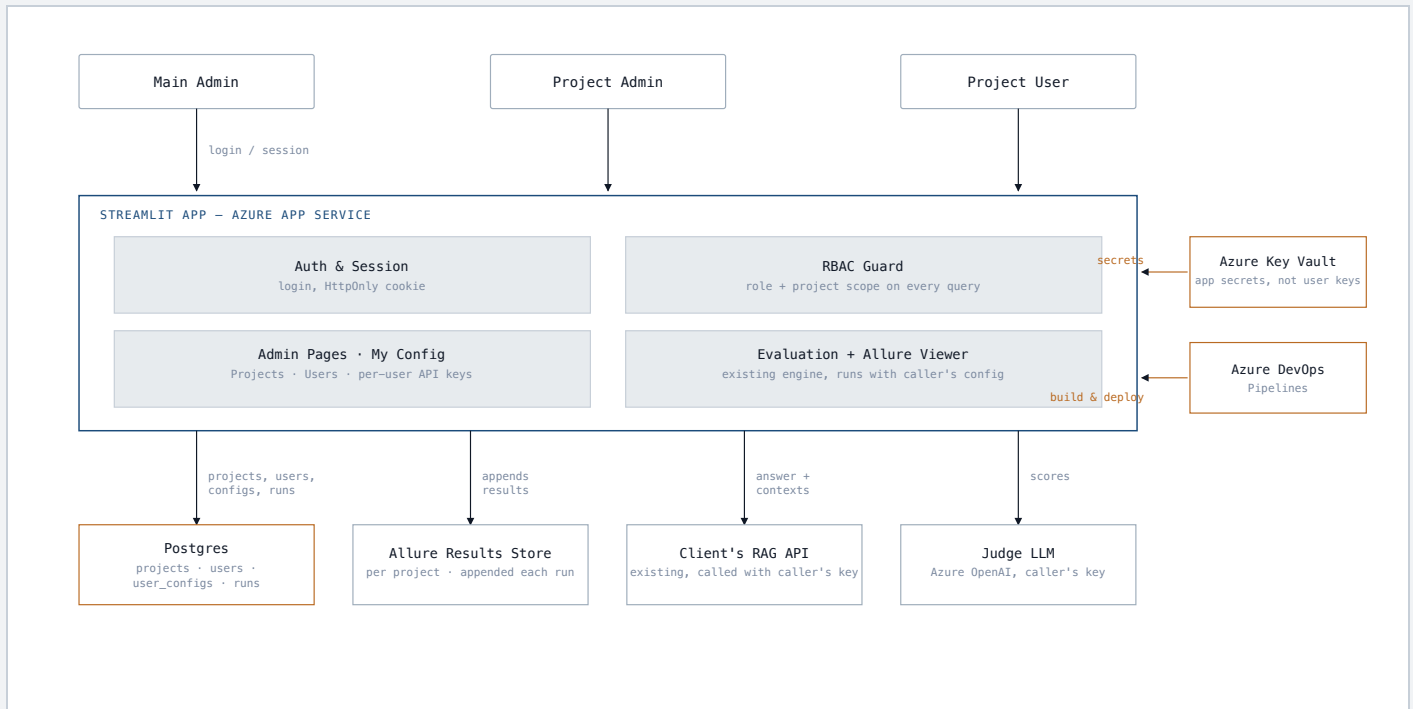


Fig. A-04 — Everything inside the accent-bordered box is one Azure App Service process. New since Rev A: "My Config" inside Admin Pages, and Postgres now also holds `user_configs` and `evaluation_runs`. Each user's own API key travels from Postgres to the RAG API / Judge LLM calls made *on their behalf* — Key Vault holds the app's own secrets, not members' personal keys (see A-07).

A-05

Data model

POSTGRES — 4 TABLES

Two tables from Rev A, plus two new ones: `user_configs` holds each member's own credentials, `evaluation_runs` records who ran what and when — the source of the "run by" tag on every appended report entry.

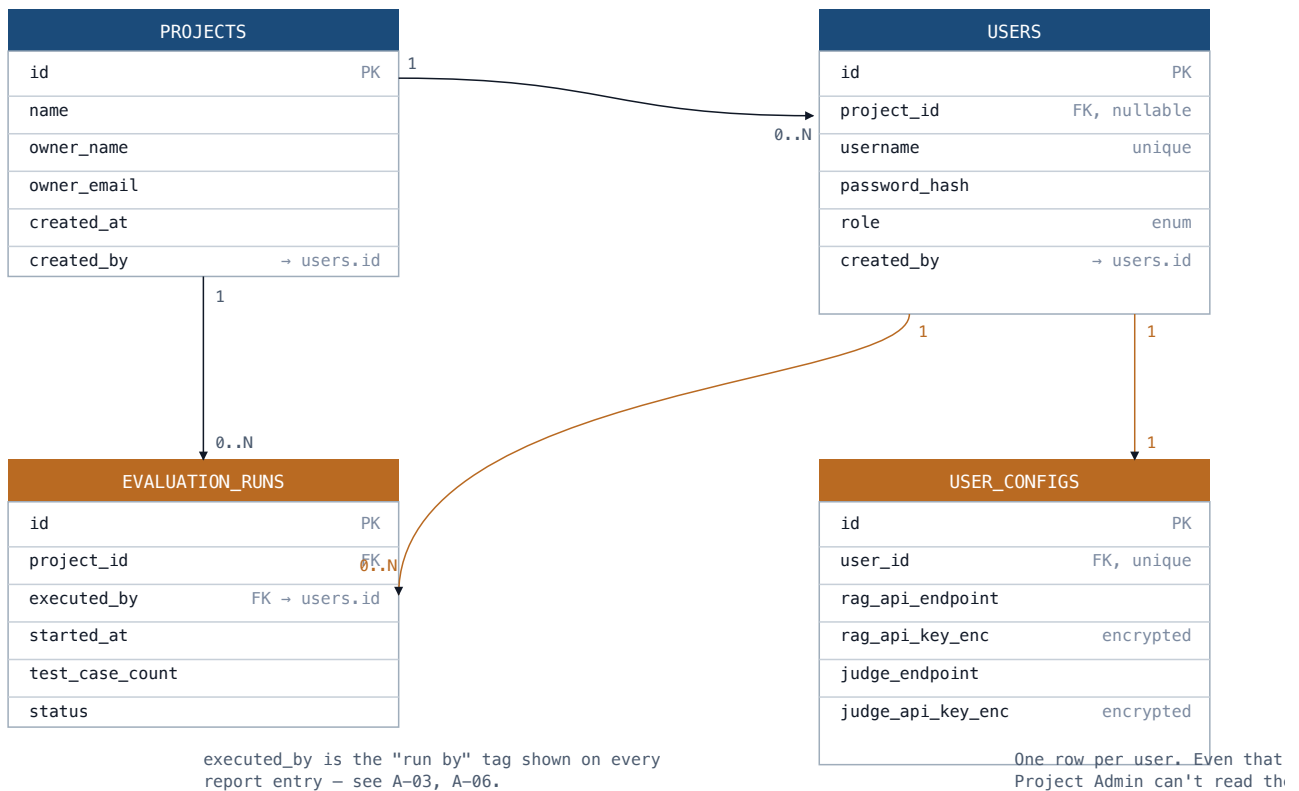


Fig. A-05 — Navy lines are Rev A's relationships; brass lines are new in Rev B. `project_id` still carries the whole permission model; `evaluation_runs.executed_by` now carries the "who ran this" answer, and `user_configs` keeps every member's credentials to themselves.

A-06

Screens

WIREFRAMES – LAYOUT & COPY ONLY

Six screens now — one new (My Config), one updated (the report gains a run history with "run by"). Sample data below is invented for illustration.

SCREEN 01 · LOGIN

Log in

No "Forgot password" yet — accounts are created directly by an admin. Future scope, see A-09.

SCREEN 02 · MAIN ADMIN DASHBOARD

4

+ Add Project

total projects

PROJECT	OWNER	USERS
Acme Retail	R. Sharma	3
Northwind Labs	T. Osei	1
Blue Harbor	M. Alavi	5

No users yet on **Acme Retail**.

Add the first login to let their team sign in.

+ Add User

"+ Add User" shown only when the viewer is that project's admin.

NAME	ROLE	ADDED	
R. Sharma	Admin	12 Jun	—
P. Nair	User	03 Jul	Remove
J. Kim	User	03 Jul	Remove

RAG API endpoint

RAG API key

Judge LLM endpoint + model

Save Config

Stored encrypted. Visible only to this signed-in user — not even their Project Admin can read it back.



RUN	RUN BY	WHEN
#12	R. Sharma	05 Aug
#11	P. Nair	03 Aug
#10	P. Nair	01 Aug

Clear previous report

Every run stays, from whoever ran it. "Clear" is Project Admin only — Users see the same history with no clear button.

A-07

Security & tenant isolation

NON-NEGOTIABLES

Data isolation

Every query is scoped by `project_id` in one shared data-access layer — not repeated per page — so a new screen can't accidentally forget to filter. A Main Admin's session is the one exception that's allowed to see across projects.

Login passwords

Stored as salted hashes (bcrypt or argon2) — never plaintext, never emailed. Whoever creates an account sets the password directly and shares it out of band.

Personal API keys NEW

Each member's saved RAG/judge-LLM key is encrypted at rest (column-level, e.g. pgcrypto) and decrypted only inside the request that runs *their* evaluation. It is never displayed again after saving, never logged, and never readable by another user — Project Admin included.

Sessions

A server-side session token in an HttpOnly cookie; role and project_id are resolved from the database on each request, never trusted from anything the browser sends.

Report isolation & append integrity NEW

Each project's Allure results live in their own storage folder. A run always appends into {project_id}/allure-results/, where project_id is resolved server-side from the acting user's own session — never from anything the client sends — so one project's history can never merge into another's, and a run can't be mis-filed even by a buggy client.

Clearing is explicit

Appending is the default on every run. Wiping a project's accumulated history is a separate, deliberate Project Admin action — never a side effect of a normal run.

Audit trail

RECOMMENDED evaluation_runs already logs who ran what and when; extend the same pattern to log who cleared a report, and when — so that action stays traceable too.

LAYER	CHOICE	WHY
App	Existing Streamlit app, extended	Reuses the evaluation engine, Allure integration and config loader already built — no rewrite risk for the pilot.
Database	Azure Database for PostgreSQL — Flexible Server	Managed Postgres, matches your stated preference; handles projects, users, per-user configs and run history comfortably.
Secret columns	pgcrypto column encryption on user_configs	Per-user API keys need to be decryptable by the app at run time, not just write-once — a Key Vault secret-per-user would work too; flagged as an open decision (A-09).
Report storage	Azure Files share, one subfolder per project	Allure's file-based output needs persistent, per-project storage that keeps growing as runs append, without re-architecting the existing writer.
App secrets	Azure Key Vault	DB credentials and the app's own service secrets stay out of source and config files — distinct from members' personal API keys above.
CI/CD	Azure DevOps Pipelines	Matches your stated preference; build → test → deploy to App Service on every merge.
Hosting	Azure App Service (Linux, container)	Simplest managed compute for a single Streamlit process; scale vertically first, horizontally later if needed.

A-09

Roadmap & open decisions

BEFORE WE BUILD

PHASE 1
PILOT

Postgres schema (all 4 tables), login & sessions, RBAC guard, Add Project / Add User, My Config, run + append, per-project isolation.

PHASE 2
HARDENING

Audit log for report-clear actions, richer dashboard (activity, last-run status per project), report history pruning policy.

PHASE 3
FUTURE SCOPE

Self-service "forgot password" / email-based reset — explicitly deferred per your note; not needed for the pilot.

OPEN DECISIONS — PLEASE CONFIRM OR CORRECT

- 01 **Bootstrapping the Project Admin.** We assumed the Main Admin sets the first Project Admin login when the project is created, using the captured owner name/email — the Project Admin doesn't self-register.
- 02 **One shared database.** We assumed a single Postgres database with row-level scoping via `project_id`, not a separate schema or database per client — simpler to run, and sufficient at this scale.
- 03 **Personal key storage.** We proposed `pgcrypto` column encryption for each member's saved API key (A-08) over a Key Vault secret per user — simpler to operate at this scale, but confirm this meets your compliance bar; Key Vault-per-user is a drop-in alternative if not.
- 04 **Report growth.** Reports now accumulate indefinitely until a Project Admin clears them. We haven't set an automatic retention limit (e.g. keep the last N runs) — confirm whether unlimited history is fine for a pilot, or a cap is wanted up front.